

CONTRACTIQ

Sprint 3 – Database Design

Contract Obligation Tracking & Compliance Management Platform

By Shrrivathsan S

1. Purpose of the Database

The ContractIQ database supports the complete contract lifecycle implemented in the attached FastAPI / SQLAlchemy / Alembic backend: contract creation and approval, document versioning, obligation tracking, renewal management, notifications, compliance evaluation, reporting, collaboration activity, and audit tracking. Each responsibility is kept in its own relational table, so information is not duplicated across modules.

This revision aligns the design with the current codebase in **contractiq.zip**: table and column names match the SQLAlchemy models and the Alembic migrations exactly (for example **contract_number** rather than `contract_code`, and **hashed_password** rather than `password_hash`), integer auto-incrementing primary keys are used throughout, and a new **compliance_records** table has been added to retain the history produced by the compliance evaluation service introduced later in the project.

The schema targets PostgreSQL and follows a consistent snake_case naming style. Timestamp, status, and foreign-key columns provide lifecycle tracking and traceability across modules.

2. Core Tables

- users – identities, roles, and account status
- contracts – master contract records and approval workflow
- contract_versions – document/version history

- obligations – duties, deadlines, and completion status
- renewals – renewal planning and outcomes
- notifications – alerts and reminders
- reports – generated report records
- compliance_records – stored compliance/risk evaluations for a contract
- audit_logs – traceability of sensitive actions
- activities – operational timeline and collaboration events

3. Proposed Schema

3.1 users

Stores platform users and access-related information.

Column	Data Type	Key / Constraint	Purpose
id	INTEGER	PK	Unique user ID
full_name	VARCHAR(150)		User's full name
email	VARCHAR(255)	UNIQUE, indexed	Login/contact email
hashed_password	VARCHAR(255)		One-way password hash
role	VARCHAR(40)	default 'Employee', indexed	Administrator, Legal Manager, Compliance Officer, Contract Manager, Department Head, Employee
is_active	BOOLEAN	default true	Account availability

Column	Data Type	Key / Constraint	Purpose
created_at	TIMESTAMP		Account creation time
updated_at	TIMESTAMP		Last profile update

3.2 contracts

Stores the main business record and approval workflow state for every contract.

Column	Data Type	Key / Constraint	Purpose
id	INTEGER	PK	Unique contract ID
title	VARCHAR(255)		Contract title
contract_number	VARCHAR(80)	UNIQUE, indexed	Internal contract reference
category	VARCHAR(80)		Service, vendor, NDA, lease, etc.
description	TEXT	nullable	Free-text contract description
start_date	DATE		Effective date
end_date	DATE		Expiry date
status	VARCHAR(30)	default 'Draft', indexed	Draft, Under Review, Approved, Active, Expired, Terminated
created_by	INTEGER	FK → users.id	User who created the contract

Column	Data Type	Key / Constraint	Purpose
assigned_to	INTEGER	FK → users.id, nullable	User currently responsible for the contract
reviewed_at	TIMESTAMP	nullable	When the contract entered review
approved_at	TIMESTAMP	nullable	When the contract was approved
created_at	TIMESTAMP		Creation timestamp
updated_at	TIMESTAMP		Last update timestamp

3.3 contract_versions

Keeps previous document versions and change history.

Column	Data Type	Key / Constraint	Purpose
id	INTEGER	PK	Unique version ID
contract_id	INTEGER	FK → contracts.id, ON DELETE CASCADE, indexed	Parent contract
version_number	INTEGER		Sequential version number

Column	Data Type	Key / Constraint	Purpose
file_path	VARCHAR(500)	nullable	Controlled storage reference for the uploaded file
change_summary	TEXT	nullable	Summary of modifications
created_by	INTEGER	FK → users.id	Uploader
created_at	TIMESTAMP		Upload timestamp

3.4 obligations

Tracks contractual duties, ownership, deadlines, and completion.

Column	Data Type	Key / Constraint	Purpose
id	INTEGER	PK	Unique obligation ID
contract_id	INTEGER	FK → contracts.id, ON DELETE CASCADE, indexed	Related contract
title	VARCHAR(255)		Short obligation title
description	TEXT	nullable	Full obligation description
obligation_type	VARCHAR(80)		Category of obligation

Column	Data Type	Key / Constraint	Purpose
due_date	DATE	indexed	Completion deadline
assigned_to	INTEGER	FK → users.id	Responsible user
status	VARCHAR(30)	default 'Pending', indexed	Pending, In Progress, Completed, Overdue (derived when a past due date is still incomplete)
completion_date	DATE	nullable	Date the obligation was completed
created_at	TIMESTAMP		Creation timestamp
updated_at	TIMESTAMP		Last update timestamp

3.5 renewals

Manages renewal schedules and renewal decisions.

Column	Data Type	Key / Constraint	Purpose
id	INTEGER	PK	Unique renewal ID
contract_id	INTEGER	FK → contracts.id, ON DELETE CASCADE, indexed	Contract being renewed

Column	Data Type	Key / Constraint	Purpose
renewal_date	DATE		Planned renewal date
previous_expiry_date	DATE		Expiry date prior to renewal
new_expiry_date	DATE	nullable	New expiry date once renewed
status	VARCHAR(30)	default 'Upcoming', indexed	Upcoming, In Progress, Renewed, Expired, Cancelled
assigned_to	INTEGER	FK → users.id, nullable	Responsible user
notes	TEXT	nullable	Renewal comments
created_at	TIMESTAMP		Record creation time
updated_at	TIMESTAMP		Last update timestamp

3.6 notifications

Stores reminders and alerts delivered to users.

Column	Data Type	Key / Constraint	Purpose
id	INTEGER	PK	Unique notification ID
user_id	INTEGER	FK → users.id, ON DELETE CASCADE, indexed	Recipient
contract_id	INTEGER	FK → contracts.id, ON DELETE SET NULL, nullable	Related contract, if applicable
obligation_id	INTEGER	FK → obligations.id, ON DELETE SET NULL, nullable	Related obligation, if applicable
notification_type	VARCHAR(50)		Deadline, renewal, overdue, system
title	VARCHAR(255)		Notification heading
message	TEXT		Notification details
status	VARCHAR(20)	default 'Unread', indexed	Unread / Read

Column	Data Type	Key / Constraint	Purpose
scheduled_at	TIMESTAMP	nullable	When the notification should fire
sent_at	TIMESTAMP	nullable	Delivery time
read_at	TIMESTAMP	nullable	When it was read
created_at	TIMESTAMP		Creation timestamp
updated_at	TIMESTAMP		Last update timestamp

3.7 reports

Stores metadata for reports produced from contract and compliance data.

Column	Data Type	Key / Constraint	Purpose
id	INTEGER	PK	Unique report ID
report_type	VARCHAR(60)		Compliance, obligations, renewals, audit
title	VARCHAR(255)		Report name
generated_by	INTEGER	FK → users.id	Report creator
file_path	VARCHAR(500)	nullable	Generated file reference
parameters	TEXT	nullable	Filters/parameters used to generate the report
created_at	TIMESTAMP		Generation timestamp

3.8 compliance_records

Retains the history produced by the compliance evaluation service, so a contract's risk posture over time can be reviewed rather than only its current status.

Column	Data Type	Key / Constraint	Purpose
id	INTEGER	PK	Unique compliance record ID
contract_id	INTEGER	FK → contracts.id, ON DELETE CASCADE, indexed	Evaluated contract
status	VARCHAR(30)		Compliant, Pending, Delayed, Non-Compliant
compliance_score	NUMERIC(5,2)		completed / total obligations × 100
risk_level	VARCHAR(20)		Low, Medium, High
completed_count	INTEGER	default 0	Completed obligations at evaluation time
pending_count	INTEGER	default 0	Pending/in-progress obligations at evaluation time

Column	Data Type	Key / Constraint	Purpose
overdue_count	INTEGER	default 0	Overdue obligations at evaluation time
notes	TEXT	nullable	Evaluator remarks
evaluated_at	TIMESTAMP		When the evaluation ran
created_at	TIMESTAMP		Record creation time
updated_at	TIMESTAMP		Last update timestamp

3.9 audit_logs

Records important changes and security-relevant actions.

Column	Data Type	Key / Constraint	Purpose
id	INTEGER	PK	Unique audit event
user_id	INTEGER	FK → users.id, ON DELETE SET NULL, nullable	Actor; nullable for system actions
action	VARCHAR(100)		create, update, delete, login, upload
entity_type	VARCHAR(80)		Affected entity/table
entity_id	INTEGER	nullable	Affected record ID
old_values	JSON	nullable	Previous state
new_values	JSON	nullable	New state

Column	Data Type	Key / Constraint	Purpose
ip_address	VARCHAR(45)	nullable	Origin IP, if captured
created_at	TIMESTAMP		Event timestamp

3.10 activities

Provides a timeline for reviews, comments, assignments, uploads, and status changes.

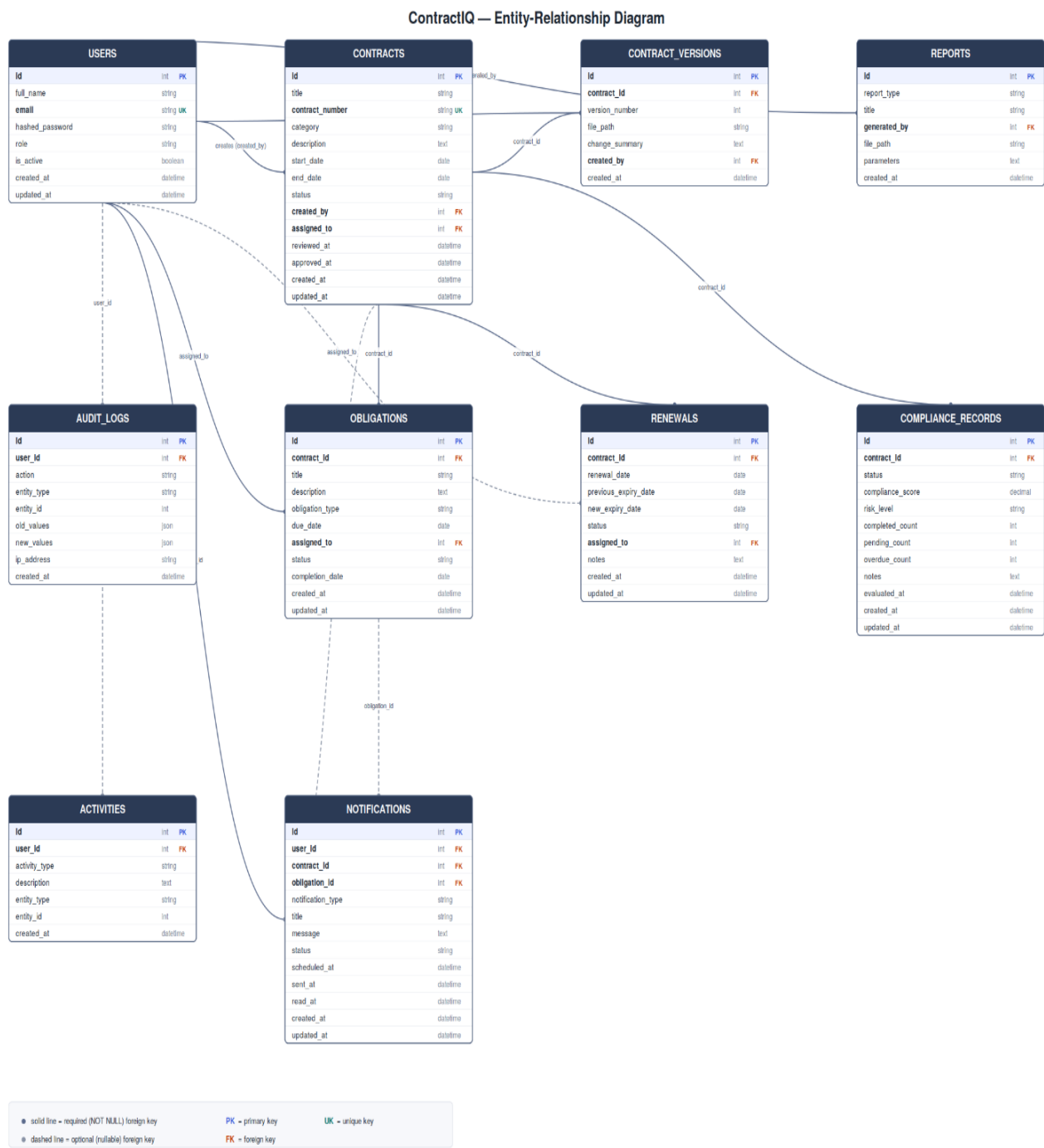
Column	Data Type	Key / Constraint	Purpose
id	INTEGER	PK	Unique activity ID
user_id	INTEGER	FK → users.id, ON DELETE SET NULL, nullable	User performing the activity
activity_type	VARCHAR(80)		comment, review, assignment, upload, status_change
description	TEXT		Human-readable activity summary
entity_type	VARCHAR(80)	nullable	Type of entity the activity relates to
entity_id	INTEGER	nullable	Related record ID
created_at	TIMESTAMP		Activity timestamp

4. Keys and Relationships

Parent	Child	Cardinality	Foreign-Key Mapping
users	contracts	One to many	users.id → contracts.created_by
users	contracts	One to many	users.id → contracts.assigned_to
users	contract_versions	One to many	users.id → contract_versions.created_by
contracts	contract_versions	One to many	contracts.id → contract_versions.contract_id
contracts	obligations	One to many	contracts.id → obligations.contract_id
users	obligations	One to many	users.id → obligations.assigned_to
contracts	renewals	One to many	contracts.id → renewals.contract_id
users	renewals	One to many	users.id → renewals.assigned_to
users	notifications	One to many	users.id → notifications.user_id
contracts	notifications	One to many	contracts.id → notifications.contract_id
obligations	notifications	One to many	obligations.id → notifications.obligation_id

Parent	Child	Cardinality	Foreign-Key Mapping
users	reports	One to many	users.id → reports.generated_by
contracts	compliance_records	One to many	contracts.id → compliance_records.contract_id
users	audit_logs	One to many	users.id → audit_logs.user_id
users	activities	One to many	users.id → activities.user_id

5. E-R Diagram:



5. Functional Interaction of the Tables

A user creates a contract, which moves through the approval workflow (**Draft** → **Under Review** → **Approved** → **Active** → **Expired/Terminated**) tracked on the `contracts.status`, `reviewed_at` and `approved_at` columns. The contract becomes the parent record for its versions, obligations, renewals, notifications, compliance evaluations, reports, and activities.

`contract_versions` preserves the evolution of the source documents. `obligations` turn contractual requirements into trackable tasks with an owner, due date, type, and status; the service layer treats an incomplete obligation with a past due_date as Overdue. `renewals` record upcoming renewal decisions and, on completion, update the associated contract's expiry date while the renewal record is retained for history.

`compliance_records` stores the output of the rule-based compliance evaluation service — status, `risk_level`, and a `compliance_score` derived from completed vs. total obligations — so a contract's risk posture can be reviewed over time rather than only in its current state. `notifications` connect users to contracts, obligations, and renewals for deadline, renewal, overdue, and system alerts. `reports` store metadata for generated compliance/obligation/renewal/audit reports. `activities` provide a collaboration timeline, while `audit_logs` capture sensitive state changes for traceability. Foreign keys connect these modules while keeping each table focused on a single responsibility.

6. Constraints and Design Decisions

- `contract_number` is unique and indexed across the system.
- `users.email` is unique and indexed for login.
- A contract version number should be unique per contract — recommended as a future `UNIQUE(contract_id, version_number)` constraint; not yet enforced at the database level in the current migrations.
- Application logic should reject an `end_date` earlier than `start_date`; this is currently validated in the service layer rather than with a database `CHECK` constraint.
- `compliance_score` is stored as `NUMERIC(5,2)` and is expected to stay within 0–100.
- Obligation, renewal, contract, notification and compliance status values are restricted through application/Pydantic validation rather than database `ENUM` types, keeping the workflow easy to extend.
- `audit_logs` should be treated as append-only from the application layer.
- Passwords are never stored in plaintext; only `users.hash_password` has persisted.
- Foreign-key columns are indexed (see the Alembic migrations) because they are frequently used for joins and filtering, along with the status columns used for dashboards and queues.
- Document references (`contract_versions.file_path`, `reports.file_path`) point to controlled storage rather than storing files directly in the database.
- `contracts.assigned_to`, `renewals.assigned_to`, `notifications.contract_id/obligation_id` and `audit_logs.user_id` are nullable with `ON DELETE SET NULL`, so history is preserved if a related user or record is removed; contract-owned child tables (`contract_versions`, `obligations`, `renewals`, `compliance_records`) cascade on contract deletion.

